



UNIVERSIDAD CATÓLICA DE SALTA
Licenciatura en Ciencia de Datos

Trabajo Práctico Integrador

Introducción a las Estructuras de Datos

Alumno: Facundo Luna

Profesor: Ing. Carlos Pastrana

Fecha: May 31, 2026

Contents

1	Análisis de grafo	2
1.1	Indicar características del grafo	2
1.2	Matriz de adyacencia	3
2	Algoritmo de pasos básicos con pilas y colas	4
3	Análisis de árbol	7
3.1	Características del árbol	7
3.2	Recorridos del árbol	8
4	Método hash de división con $m = 50$	9

1. Análisis de grafo

Dado el siguiente grafo:

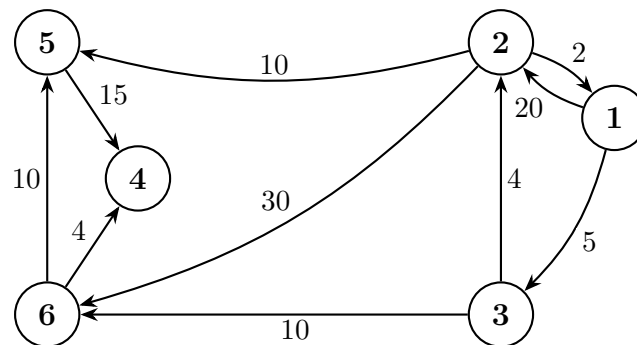


Figure 1: Grafo a analizar

1.1. Indicar características del grafo

a) Indique:

- ¿Es un grafo dirigido, no dirigido?
- ¿Es conexo?
- ¿Es ponderado?
- ¿Maximal? ¿Minimal?

Solución:

- **Dirigido o no dirigido:** Es un grafo **dirigido**, ya que cada arista tiene una dirección definida por una flecha.
- **Conexo:** El grafo es **débilmente conexo**: ignorando las direcciones de las aristas, el grafo subyacente es conexo. Sin embargo, **no es fuertemente conexo**, ya que el nodo 4 no posee aristas de salida, por lo que no es posible alcanzar ningún otro nodo partiendo desde él.
- **Ponderado:** El grafo **es ponderado**, ya que cada arista tiene un peso numérico asociado.
- **Maximal / Minimal:** El vértice 4 es maximal porque no posee aristas de salida. No existen vértices minimales, ya que todos los nodos tienen al menos una arista de entrada.

1.2. Matriz de adyacencia

b) *Represente el grafo mediante una matriz de adyacencia.*

Solución:

La matriz de adyacencia A para los nodos $\{1, 2, 3, 4, 5, 6\}$ es:

$$A = \begin{pmatrix} & 1 & 2 & 3 & 4 & 5 & 6 \\ 1 & 0 & 20 & 5 & 0 & 0 & 0 \\ 2 & 2 & 0 & 0 & 0 & 10 & 30 \\ 3 & 0 & 4 & 0 & 0 & 0 & 10 \\ 4 & 0 & 0 & 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 15 & 0 & 0 \\ 6 & 0 & 0 & 0 & 4 & 10 & 0 \end{pmatrix}$$

donde $A_{ij} = w$ indica que existe una arista dirigida desde el nodo i hacia el nodo j con peso w , y $A_{ij} = 0$ indica su ausencia.

[↑ Volver al índice](#)

2. Algoritmo de pasos básicos con pilas y colas

Desarrollar un algoritmo de pasos básicos que me permita recorrer una lista y:

a) Al encontrarse un elemento par, deberá generar y mostrar una pila; caso distinto, al encontrarse con un elemento impar, deberá generar y mostrar una cola resultante conteniendo los números impares.

b) Mostrar el promedio de los elementos de la pila P1.

Solución:

El algoritmo valida primero que todos los elementos sean enteros. Si hay alguno inválido, informa el error y termina. De lo contrario, recorre la lista y apila los pares o encola los impares según corresponda, para luego calcular el promedio de la pila.

```
1 import java.util.ArrayDeque;
2 import java.util.Arrays;
3 import java.util.Deque;
4
5 public class Ejercicio02 {
6     public static void main(String[] args) {
7         // lista hardcodeada por simplicidad
8         Object[] list = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
9         for (Object elemento : list) {
10             // no reservo memoria si hay un elemento inválido
11             if (!(elemento instanceof Integer)) {
12                 System.out.println("Error: ' " + elemento + "' no
13                     es un número entero.");
14                 return;
15             }
16
17             Integer suma = 0;
18             // tipo Deque (interfaz) para no acoplar el código a una
19             // implementación
20             Deque<Integer> pila = new ArrayDeque<>();
21             Deque<Integer> cola = new ArrayDeque<>();
22
23             for (Object elemento : list) {
24                 Integer numero = (Integer) elemento;
25                 (numero % 2 == 0 ? pila : cola).add(numero);
26             }
27             System.out.println("item a");
28             System.out.println("Lista original: " +
29                 Arrays.toString(list));
30             System.out.println("Pila Auxiliar (pares): " + pila);
31             System.out.println("Cola Auxiliar (impares): " + cola);
32
33             for (Integer par : pila) {
34                 suma += par; // acumulo para luego dividir por el
35                             // tamaño de la pila
36             }
37             System.out.println("item b");
38             System.out.println("El Promedio de la Pila Auxiliar es: "
39                 + (suma / (double) pila.size()));
40         }
41     }
42 }
```

Salida — caso exitoso (todos los elementos son enteros):

```
item a)
Lista original: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
Pila Auxiliar (pares): [0, 2, 4, 6, 8]
Cola Auxiliar (impares): [1, 3, 5, 7, 9]
item b)
El Promedio de la Pila Auxiliar es: 4.0
```

Salida — caso de error (la lista contiene un elemento no entero, p.ej. "hola" en la posición 2):

```
Error: 'hola' no es un número entero.
```

[↑ Volver al índice](#)

3. Análisis de árbol

Dado el siguiente árbol:

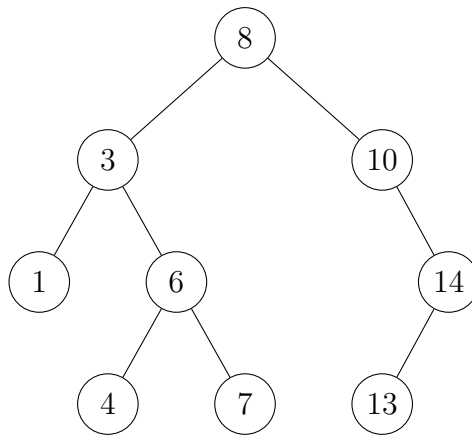


Figure 2: Árbol binario a analizar

3.1. Características del árbol

a) Indicar:

- ¿Qué tipo de árbol es? ¿Es completo?
- ¿Altura?
- ¿Se encuentra balanceado? Fundamente

Solución:

- **Tipo de árbol:** Es un **árbol binario de búsqueda (ABB)**. No es **completo**, ya que el nodo 10 no tiene hijo izquierdo, lo que rompe el requisito de que los nodos del último nivel estén alineados a la izquierda.
- **Altura:** Considerando la raíz con altura 1, el árbol tiene **altura 4**.
- **Balanceo:** El árbol **no está balanceado**. Para determinarlo se calcula el Factor de Balance (FB) de cada nodo, definido como:

$$FB(n) = altura_subárbol_izquierdo(n) - altura_subárbol_derecho(n)$$

Los factores de balance obtenidos son:

Nodo	FB
1	0
4	0
7	0
13	0
6	0
14	-1
3	-1
10	-2
8	0

El nodo 10 tiene $FB = -2$, valor fuera del rango $[-1, 1]$ exigido por un árbol AVL. Por lo tanto, el árbol se considera **desbalanceado**.

3.2. Recorridos del árbol

b) Generar los recorridos “pre order”, “in order”, “post order” del árbol dado.

Solución:

- **In-order** (izquierda $\rightarrow$ raíz $\rightarrow$ derecha):

1, 3, 4, 6, 7, 8, 10, 13, 14

- **Pre-order** (raíz $\rightarrow$ izquierda $\rightarrow$ derecha):

8, 3, 1, 6, 4, 7, 10, 14, 13

- **Post-order** (izquierda $\rightarrow$ derecha $\rightarrow$ raíz):

1, 4, 7, 6, 3, 13, 14, 10, 8

[↑ Volver al índice](#)

4. Método hash de división con $m = 50$

Por el método hash de división con $m = 50$, obtener los valores hash de los siguientes conjuntos de claves: ‘Ariel’, ‘Luis’, ‘Gonzalo’, ‘Federico’, ‘Roberto’, ‘Carlos’.

Solución:

Para cada clave se suman los valores ASCII de sus caracteres y se aplica el módulo $m = 50$. La tabla siguiente detalla el cálculo completo.

Clave	Suma de valores ASCII	Suma	$h(k)$
Ariel	$65 + 114 + 105 + 101 + 108$	493	$493 \bmod 50 = \mathbf{43}$
Luis	$76 + 117 + 105 + 115$	413	$413 \bmod 50 = \mathbf{13}$
Gonzalo	$71 + 111 + 110 + 122 + 97 + 108 + 111$	730	$730 \bmod 50 = \mathbf{30}$
Federico	$70 + 101 + 100 + 101 + 114 + 105 + 99 + 111$	801	$801 \bmod 50 = \mathbf{1}$
Roberto	$82 + 111 + 98 + 101 + 114 + 116 + 111$	733	$733 \bmod 50 = \mathbf{33}$
Carlos	$67 + 97 + 114 + 108 + 111 + 115$	612	$612 \bmod 50 = \mathbf{12}$

No se presentan colisiones entre las claves dadas: los seis valores hash obtenidos (43, 13, 30, 1, 33, 12) son todos distintos.

[↑ Volver al índice](#)